Pimpri Chinchwad College of Engineering
Department of MCA

Advanced Web Technologies — Mini Project
Formative Assessment II (FA-2): Advanced Features, Testing and Deployment

✦ Recall

Digital Flashcard Learning Application with Spaced Repetition

Submitted by
Ishita Khandelwal
PRN: 125M1H044 · Class: SYMCA · Semester III

Course Owner: Dr. Avinash Chormale
Module Coordinator: Dr. Rajkamal Sangole

Academic Year 2026–27

Abstract

Students forget most of what they read within days unless they revisit it at the right moment. **Recall** is a full-stack web application that turns study material into flashcards and uses the **SM-2 spaced-repetition algorithm** to decide when each card should be reviewed again. Phase I (FA-1) delivered the React user interface with local state. Phase II (FA-2), described in this report, adds a REST backend (Node.js, Express 5, Prisma ORM, SQLite), client-side routing, validated forms, API integration with full CRUD, search and filtering, robust error handling, performance optimisation, a progress dashboard, 76 automated tests plus an end-to-end browser test, and deployment configuration.

1. Introduction and Problem Statement

Re-reading notes is the most common study technique and one of the least effective. Research on memory (the "forgetting curve") shows that recall improves most when information is actively retrieved just before it would be forgotten. Paper flashcards support active recall but give no guidance on **what** to review **when**, and existing tools are often cluttered or paywalled.

Problem statement: Build a clean, responsive web application where learners can create decks of flashcards, study them with active recall, and have the system automatically schedule each card for review based on how well they remembered it, while tracking progress over time.

2. Objectives

- Let users create, organise, search, edit and delete decks and flashcards (full CRUD) stored on a server.
- Provide an engaging study mode with a flip-card interaction, keyboard shortcuts and four-level self-grading.
- Schedule reviews automatically with the SM-2 algorithm so that "due" cards surface at the right time.
- Visualise progress: streaks, recall rate, daily activity and mastery per deck.
- Apply advanced React concepts (hooks, custom hooks, context, reducers, routing, lazy loading) and good engineering practice (validation on both tiers, consistent error handling, automated testing, CI and deployment).

3. Recap of Phase I (FA-1)

Phase I produced a single-page React app with components `DeckList`, `DeckCard`, `DeckDetail`, `CardForm`, `StudyView` and `Flashcard`. It demonstrated JSX, props, `useState`, event handling, conditional rendering, lists with keys and a responsive layout. Data lived in `localStorage` and navigation was a `view` state variable. Phase II keeps that visual design and those concepts, and replaces the storage and navigation layers with a real backend and router.

4. Requirement Analysis

4.1 Functional requirements

ID	Requirement
FR1	Create, view, edit and delete decks (title, subject, description, colour).
FR2	Add, edit and delete cards (question/answer) inside a deck.
FR3	Search decks by title, subject, description or card content; filter by subject; sort four ways.
FR4	Search and filter cards in a deck by status: New, Learning, Mastered, Due.
FR5	Study all cards or only due cards, optionally shuffled; flip and grade each card

ID	Requirement
	Again/Hard/Good/Easy.
FR6	Reschedule each graded card with SM-2 and record the review.
FR7	Show a session summary and allow retrying only the missed cards.
FR8	Show a progress dashboard: streak, reviews today, due count, recall rate, 14-day activity, mastery per deck, recent sessions.
FR9	Import and export decks as JSON; reset a deck's progress.
FR10	Validate all input on client and server with clear, field-level messages.

4.2 Non-functional requirements

- **Usability & accessibility:** keyboard operable, semantic HTML, ARIA labels, visible focus, reduced-motion support, dark mode.
- **Responsiveness:** usable from 375 px phones to desktop with no horizontal scrolling.
- **Reliability:** friendly handling of server downtime, timeouts, 404s and unexpected UI crashes.
- **Performance:** code-split routes, memoised components and debounced network calls.
- **Maintainability:** modular folders, reusable hooks and components, automated tests and CI.

4.3 Use cases

Actor	Use case	Outcome
Learner	Create a deck and add cards	Deck saved on server; card form stays open for fast entry
Learner	Review due cards	Cards rescheduled; session saved to stats
Learner	Find a deck or card	Debounced search shows matching results; URL keeps filters
Learner	Track progress	Dashboard shows streak, accuracy and mastery
Learner	Share / back up a deck	Export to JSON; import on another device

5. System Architecture

Recall follows a classic three-tier architecture. The React single-page application talks to an Express REST API over JSON; the API uses Prisma ORM to read and write a SQLite database. In development, Vite proxies `/api` to the backend; in production the frontend reads the API origin from the `VITE_API_URL` environment variable.

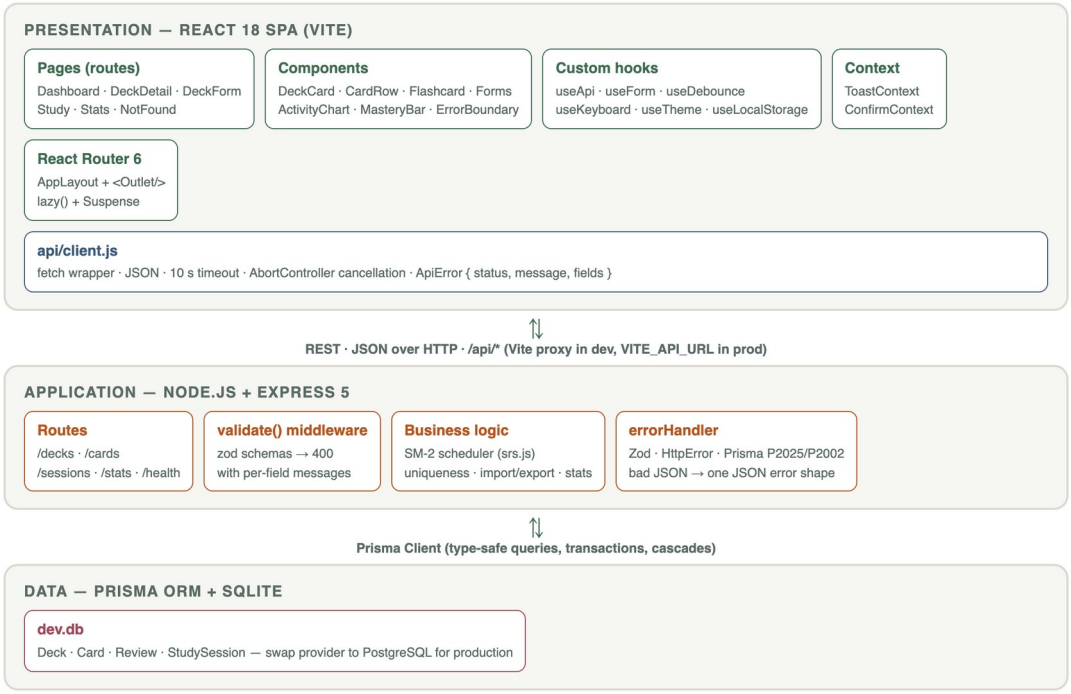


Figure 1 — Three-tier architecture of Recall

5.1 Technology stack

Layer	Technology	Why
UI	React 18, React Router 6, Vite 5	Component model, hooks, fast dev server and optimised builds
Styling	Plain CSS with custom properties	Theming (light/dark) via tokens; no framework lock-in
API	Node.js, Express 5	Minimal, familiar; Express 5 forwards async errors automatically
Validation	zod (server) + custom validators (client)	Same rules on both tiers; typed, composable schemas
Database	Prisma ORM 6 + SQLite	Type-safe queries, schema-first modelling, zero-install DB; swap to PostgreSQL by changing one line
Testing	Vitest, React Testing Library, Supertest, Playwright	Fast unit/component/API tests and real-browser E2E
DevOps	GitHub Actions, Vercel/Netlify, Render	CI on every push; free hosting targets

6. Database Design

The schema is defined in backend/prisma/schema.prisma. A **Deck** has many **Cards**; each Card stores its spaced-repetition state and has many **Review** rows (one per answer). **StudySession** rows summarise each completed session. All child relations use onDelete: Cascade, so deleting a deck cleanly removes its cards, reviews and sessions. Indexes on deckId, dueAt, reviewedAt and completedAt keep the common queries fast.

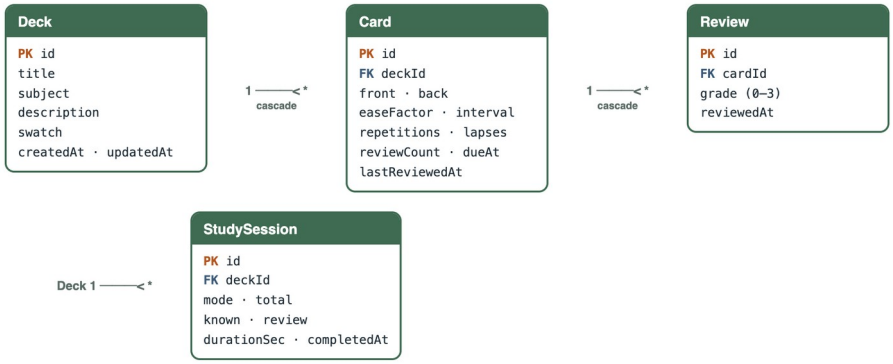


Figure 2 — Entity-relationship diagram

7. Implementation of Advanced Features

7.1 React Hooks and custom hooks

Built-in hooks used: useState, useEffect, useReducer, useContext, useMemo, useCallback, useRef, useId and forwardRef. Seven custom hooks encapsulate reusable logic:

Hook	Responsibility
useApi(fetcher, deps)	Loads data, tracks loading/error, aborts stale requests, exposes reload() and setData() for optimistic updates
useForm({ initialValues, validate, onSubmit })	Controlled values, touched state, client + server field errors, submitting flag, reset
useDebounce(value, delay)	Delays search input so the API is called once per pause in typing
useKeyboard(bindings)	Global shortcuts that ignore typing in inputs and preserve native button behaviour
useLocalStorage / useTheme	Persisted preferences (shuffle, light/dark theme)
useDocumentTitle	Per-page browser tab titles

The study session is a small state machine implemented with **`useReducer`** (start, flip, grade actions), which keeps transitions pure and unit-testable. **Context** provides app-wide toast notifications and a promise-based confirm dialog (await confirm({...})) that replaces the blocking window.confirm.

7.2 Routing

Path	Page
/	Dashboard — decks with search, subject filter and sort (state mirrored in ?q=&subject=&sort=)
/decks/new, /decks/:deckId/edit	Create / edit deck (one reusable form)
/decks/:deckId	Deck detail — cards, filters, add/edit/delete, export, reset
/decks/:deckId/study?mode=due&shuffle=1	Study session
/stats	Progress dashboard
*	404 Not Found

All routes share AppLayout, which renders the navbar, a server-status banner and an <Outlet /> wrapped in an ErrorBoundary and Suspense. NavLink highlights the active section; useNavigate handles redirects after create/delete; location.state opens the card form automatically after a new deck is created.

7.3 Forms and validation

Validation happens twice. On the client, utils/validators.js gives instant feedback: required fields, length limits (title 2–60, subject ≤30 with an allowed-character pattern, question 2–300, answer ≤500) and a rule that the answer must differ from the question. Errors appear after a field is blurred or the form is submitted, and every field shows a live character counter. On the server, zod schemas enforce the same rules and a validate() middleware returns **400** with a fields map. Business rules that need the database — unique deck titles and unique questions within a deck (case-insensitive) — return **409 Conflict** with a field message, which useForm places under the correct input.

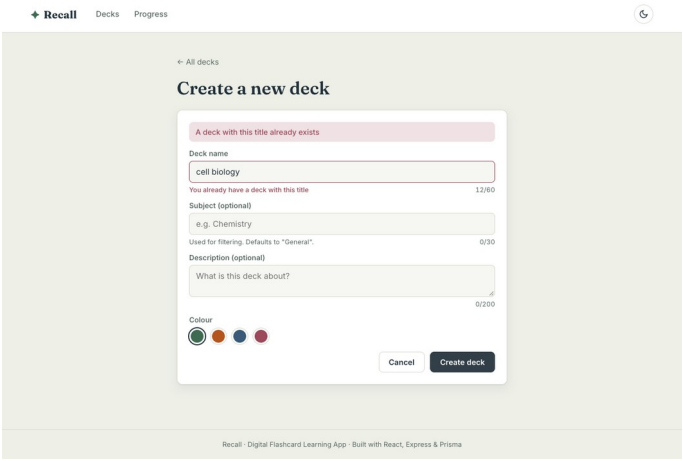


Figure 3 — Client-side and server-side (409) validation messages

7.4 API integration and CRUD

src/api/client.js wraps fetch: it builds query strings, sends JSON, applies a 10-second timeout, links the caller's AbortSignal, handles 204 No Content, and converts every failure into an ApiError with status, message and optional fields. Endpoint functions are grouped in decksApi, cardsApi, sessionsApi and statsApi. The backend exposes 20 endpoints (see README), covering create/read/update/delete for decks and cards plus study, review, sessions, stats, import/export and reset. After a mutation the UI updates immediately from the server response (for example recomputing deck counts locally) instead of reloading the page.

7.5 Search and filter

- **Decks (server-side):** a debounced search box queries GET /decks?q= which matches titles, subjects, descriptions and card text. Subject chips and a sort menu (recent, A–Z, most cards, most due) combine with the search. Filters are mirrored into the URL so a reload or shared link restores them; the / key focuses search.
- **Cards (client-side):** within a deck, text search and status chips (All / Due / New / Learning / Mastered, each with a count) are computed with useMemo over already-loaded data.

7.6 Error handling

- **Server:** one errorHandler middleware maps zod errors → 400, HttpError → its status, Prisma P2025 → 404, P2002 → 409, malformed JSON → 400, oversize body → 413, and anything else → 500 with the stack logged. Unknown routes return a JSON 404.
- **Client:** a class ErrorBoundary (keyed by route) catches render crashes; pages show error states with a **Try again** button; a banner appears when /api/health is unreachable; mutation failures raise toast

notifications; unknown URLs and deleted decks render a 404 page; answers are saved fire-and-forget so a slow network never blocks studying.

7.7 Spaced repetition (SM-2)

Each grade maps to SM-2 quality 1/3/4/5. The ease factor is updated with EF′ = EF + (0.1 − (5−q)(0.08 + (5−q)·0.02)), floored at 1.3. "Again" resets repetitions and brings the card back in 10 minutes; otherwise the interval grows 1 → 6 → interval × EF days, with ×0.8 for Hard and ×1.3 for Easy. A card is **mastered** once its interval reaches 7 days. The scheduler is a pure function (`backend/src/lib/srs.js`) and a mirror on the client previews the next interval on each grade button.

7.8 Responsive design and accessibility

Layouts use CSS Grid (`auto-fill, minmax(240px, 1fr)`) and Flexbox with `clamp()` spacing; below 640 px the grids collapse to one column, grade buttons become a 2×2 grid and toasts span the screen. The flashcard is a real `<button>` with descriptive ARIA labels, dialogs use `role="alertdialog"` with focus management, and a skip-link, visible focus rings and `prefers-reduced-motion` support are included. Dark mode is a separate set of colour tokens, applied before first paint to avoid flashing.

7.9 Performance optimisation

Technique	Where	Effect
React.lazy + Suspense	All pages except the dashboard	Initial JS ≈ 22.7 kB app + 164 kB cached vendor chunk; each page 0.5–7.8 kB loaded on demand
manualChunks	vite.config.js	React/Router in a long-cached vendor bundle
React.memo	DeckCard, CardRow	List items skip re-render while typing in search
useMemo / useCallback	Card filtering, handlers	No recomputation or prop churn on unrelated renders
Debounce (300 ms)	Deck search	One request per pause instead of per keystroke
AbortController	useApi	Out-of-order responses discarded; bandwidth saved
DB indexes, select	Prisma queries	Only needed columns fetched for counts

8. Testing

Testing is layered so that each level catches different faults. All 76 automated tests pass (`npm run test:all`), and the end-to-end walkthrough passes with no console errors.

Suite	Tool	Tests	What is covered
Backend unit	Vitest	11	SM-2 intervals, ease floor, lapses, status helpers, streak calculation
Backend API	Vitest + Supertest	20	CRUD for decks/cards, 400/404/409 responses, bad JSON, search/filter/sort, review scheduling, reset, import/export, sessions, stats
Frontend unit	Vitest	18	Validators, API client (timeouts, network errors, field errors), SM-2 preview
Hooks	Vitest + RTL	6	Debounce with fake timers, form state & server

Suite	Tool	Tests	What is covered
	renderHook		errors, request cancellation
Components & pages	React Testing Library	21	Forms, flashcard, deck card, chips, error boundary, dashboard search/filter/delete, full keyboard study session
End-to-end	Playwright + Chrome	17 steps	Real browser against real servers: create → validate → add/edit/delete → study → stats → dark mode → 404 → mobile → import

API tests run against a separate throwaway `test.db` and refuse to start against any other database. During development, the E2E test exposed two real bugs that were fixed: a form race where typing during a save could be wiped by the reset, and stale URL parameters when changing two filters in quick succession. A GitHub Actions workflow runs both suites and a production build on every push.

9. Deployment

- **Frontend** — Vercel (`vercel.json`) or Netlify (`public/_redirects`) with an SPA fallback so deep links work; set `VITE_API_URL` to the API origin.
- **Backend** — `render.yaml` blueprint with health check `/api/health`; `CORS_ORIGIN` restricts which frontends may call the API. For persistent production data the Prisma provider can be switched to PostgreSQL without code changes.
- **Local** — `npm run setup` installs everything, creates the database and seeds sample decks with study history; `npm run dev:all` starts both servers.

10. Results — Screenshots

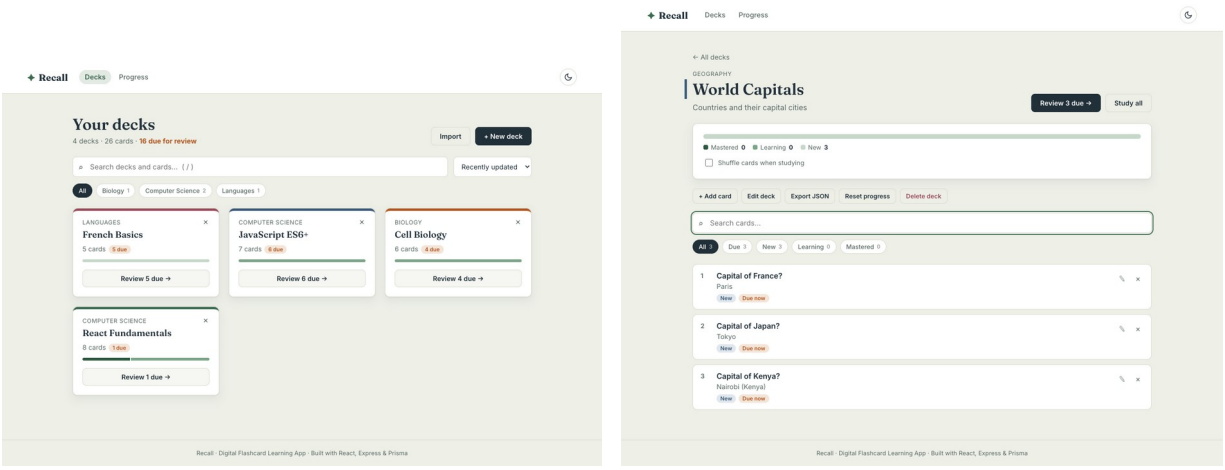
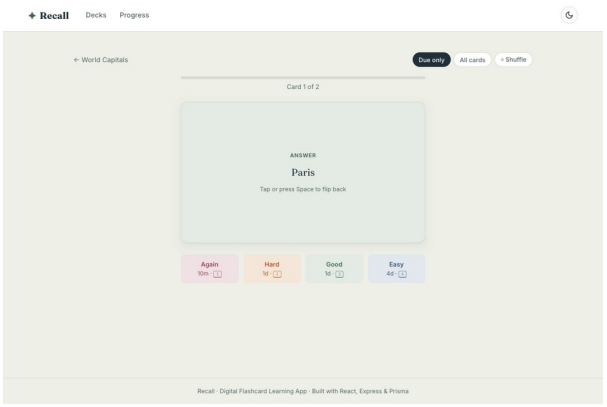


Figure 4 — Dashboard (search, subject filter, due counts) · Figure 5 — Deck detail (mastery, card filters)



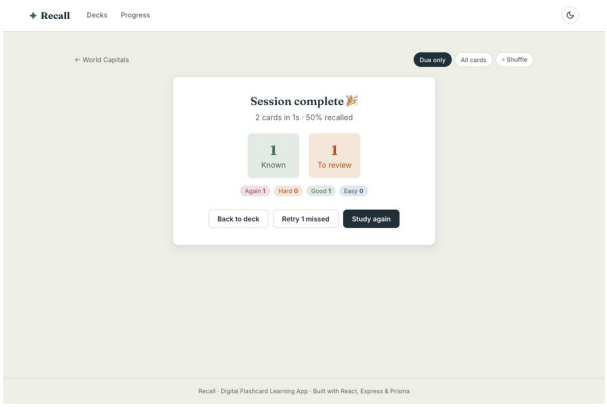
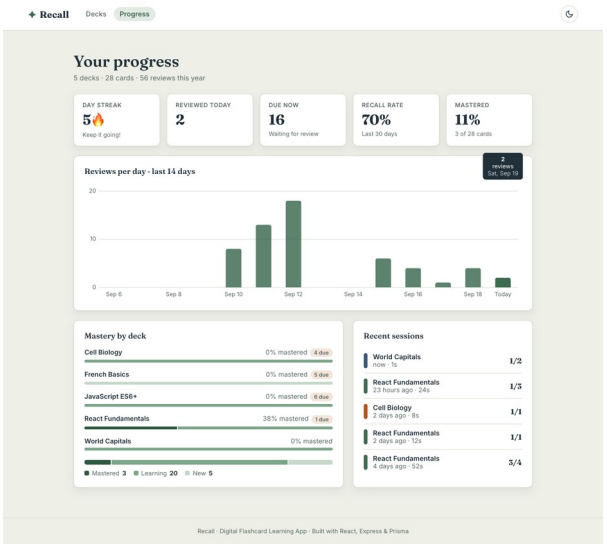


Figure 6 — Study mode with SM-2 interval previews · Figure 7 — Session summary with retry-missed



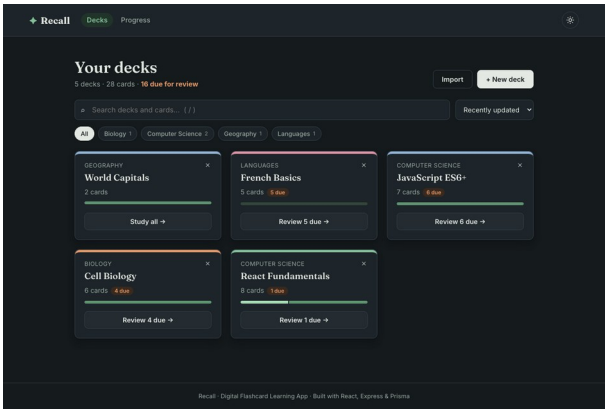


Figure 8 — Progress dashboard · Figure 9 — Dark mode

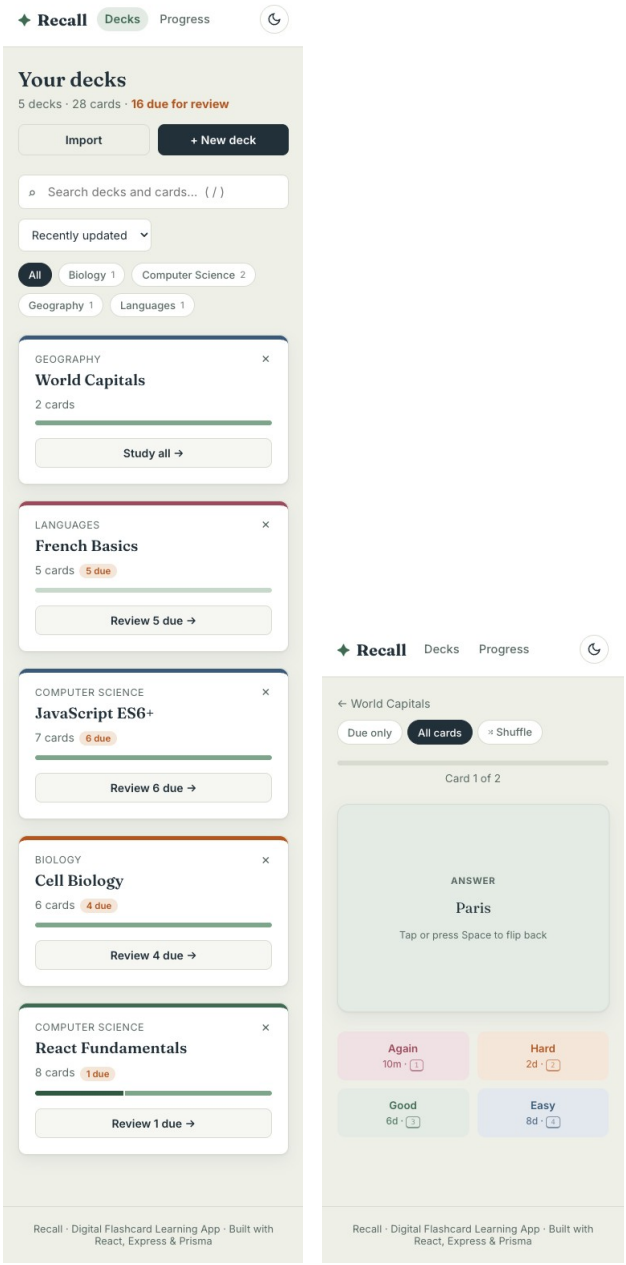


Figure 10 — Mobile layouts at 375 px

11. Conclusion and Future Scope

Recall evolved from a local-state React prototype into a complete, tested, deployable full-stack application. Phase II met every item in the FA-2 scope — hooks, routing, forms and validation, API integration, CRUD, search and filter, error handling, responsive design, performance optimisation, testing and deployment — and went further with spaced-repetition scheduling, analytics, import/export, keyboard control, dark mode and accessibility work.

Future scope: user accounts with authentication (JWT) so decks are private per user; images and audio on cards; sharing public decks; offline support with a service worker (PWA) and background sync; AI-assisted card generation from notes; and PostgreSQL in production.

References

- React documentation — https://react.dev
- React Router v6 documentation — https://reactrouter.com/v6

- Express 5 documentation — https://expressjs.com
- Prisma ORM documentation — https://www.prisma.io/docs
- Zod documentation — https://zod.dev
- Vitest and Testing Library — https://vitest.dev, https://testing-library.com
- P. A. Woźniak, "SuperMemo 2 algorithm" (1990) — https://super-memory.com/english/ol/sm2.htm